

Quem somos?

Mais de 75.000
participantes satisfeitos



Um catálogo com
mais de 6.000 cursos



Mais de 7.150
instrutores certificados



Presença em
81 países



Mais de 20 anos
de experiência



O que nos torna únicos?



Soluções sob medida



Programas abrangentes



Atendimento personalizado



Instrutores altamente
qualificados



Cursos práticos e mão
na massa



Turmas pequenas



Por que nos escolher novamente?



Impulsionamos o seu sucesso

Oferecemos soluções completas de treinamento e consultoria para ajudar você a alcançar seus objetivos.



Presença global

Estamos onde você está. Atendimento e representação local com alcance global.



Flexibilidade total

Adaptamos nossos serviços às suas necessidades, entregando exatamente o que você precisa.



Otimização garantida

Ajudamos você a otimizar seus processos de treinamento para obter resultados superiores.



Resposta rápida

Receba suporte rápido e eficiente, sempre!

NobleProg

Descubra como desbloquear seu potencial com nossos cursos especializados de treinamento!

Visite nosso site, selecione seu país e encontre a solução de treinamento perfeita para sua equipe.

www.nobleprog.com.br



NobleProg

PROGRAMA DE TREINAMENTO

TestComplete

Instrutor: Jefferson Bruno

Data: 02/06/2026 | Duração: 21 horas

Formato: Online

The World's
Local Training
Provider



www.nobleprog.com.br



Instrutor: Jefferson Bruno

Meu Nome é Jefferson, tenho 38 anos, sou cristão, casado, pai de três filhos.

Sou profissional de Tecnologia da Informação com mais de **15 anos**, nos últimos anos dedicados à área de **Qualidade de Software (QA)**, atuando com testes funcionais, automação de testes, BDD, testes de API e performance. Possuo experiência em projetos de diferentes segmentos, participando desde o levantamento de requisitos até a validação e garantia da qualidade das entregas.

Atualmente atuo como **QA**, com mais de **12 anos de experiência na TOTVS**, trabalhando com análise de requisitos, criação de cenários de testes, automação utilizando **TestComplete** e **Cypress**, além de integração com processos de **CI/CD** e metodologias ágeis.

Ao longo da minha carreira, também participei de projetos para empresas como Nestlé, Suzano e DM Card, contribuindo para a evolução da qualidade de produtos e sistemas. Minha missão neste curso é compartilhar conhecimentos práticos, experiências reais do mercado e boas práticas de automação de testes com **TestComplete**, ajudando profissionais a desenvolverem soluções mais confiáveis, eficientes e de alta qualidade.

Programação — Conteúdo do Curso

01

Módulo 1: Controle de Versão com Git

02

Módulo 2: Lógica de Programação (Base)

03

Módulo 3: Arquitetura do Projeto TC

04

Módulo 4: Introdução ao DelphiScript

05

Módulo 5: Estruturas de Controle

06

Módulo 6: Planilhas & Banco de Dados

07

Módulo 7: Métodos & Propriedades do TC

08

Módulo 8: Boas Práticas & Encerramento

Controle de Versão com Git

O que é Git e por que usar?

NobleProg

Git é um sistema de controle de versão distribuído que registra o histórico de alterações de arquivos ao longo do tempo.

Histórico

Cada alteração é gravada com data, autor e mensagem.

Colaboração

Múltiplos devs trabalham em paralelo sem conflito.

Segurança

Nada é perdido — qualquer versão pode ser restaurada.

Branches

Isola novas features do código estável (main).

Instalação & Configuração Inicial

1. Instale o Git: <https://git-scm.com/downloads>

```
# Identifique-se (feito uma só vez)
git config --global user.name "Seu Nome"
git config --global user.email "voce@email.com"

# Verifique a configuração
git config --list
```

2. Criando seu primeiro repositório

```
mkdir meu-projeto
cd meu-projeto
git init          # Inicializa repositório local
git status        # Mostra o estado atual da área de trabalho
```

Fluxo Fundamental: add → commit → push

```
# 1. Ver o que mudou
git status

# 2. Adicionar arquivo(s) à área de stage
git add meuArquivo.sd      # arquivo específico
git add .                  # todos os arquivos alterados

# 3. Registrar o snapshot com mensagem
git commit -m "feat: adiciona cenário CT_01 para rotina X"

# 4. Enviar para o repositório remoto
git push origin main
```

Working
Directory



Stage
(add)



Repositório
Local (commit)



Repositório
Remoto (push)

Branches: Trabalhando em Paralelo

```
# Listar branches
git branch

# Criar e já ir para a branch nova
git checkout -b feature/CT-15-rotina-1122

# Voltar para a branch principal
git checkout main

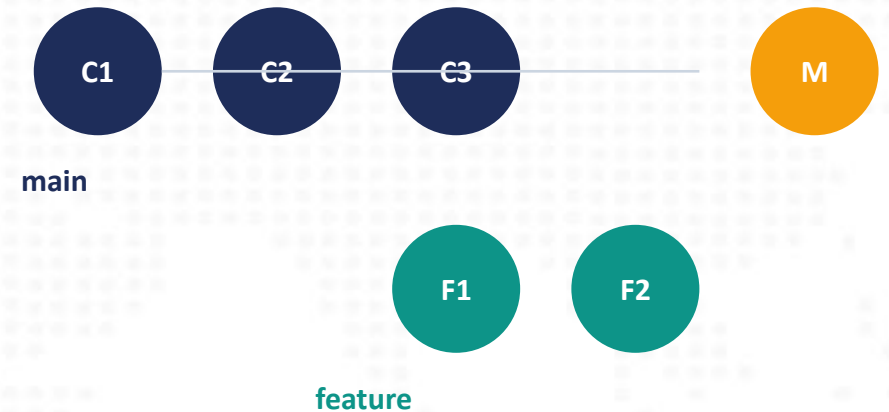
# Incorporar as alterações da feature ao main
git merge feature/CT-15-rotina-1122

# Deletar a branch após merge
git branch -d feature/CT-15-rotina-1122
```

Comandos úteis extras

```
git pull          # Baixar atualizações do remoto
git log --oneline  # Ver histórico resumido
git diff          # Ver diferenças não commitadas
git stash         # Guardar mudanças temporariamente
```

Diagrama



Boas Práticas — Mensagens de Commit

Siga o padrão Conventional Commits para manter o histórico legível:

```
feat: adiciona validação de campo CUSTOFIN  
fix: corrige separador decimal Oracle (vírgula → ponto)  
test: cria CT_03 para cancelamento de pedido  
docs: atualiza README com instrução de ODBC
```

✓ **Faça** Commits pequenos e frequentes; mensagem descritiva no imperativo.

✓ **Faça** Crie uma branch por funcionalidade ou caso de teste.

✗ **Evite** Commitar código quebrado na branch main.

✗ **Evite** Mensagens vazias ou genéricas como 'ajustes' ou 'teste'.

Logica de Programação

Fundamentos: Variáveis e Tipos de Dados

Uma variável é um espaço nomeado na memória para guardar um valor.

```
// DelphiScript – declaração de variáveis  
var nome      : string;    // texto livre  
var idade     : integer;   // número inteiro (-2.1bi a +2.1bi)  
var preco     : double;    // número real (Float/Double)  
var ativo     : boolean;   // verdadeiro ou falso  
var dataVenda : TDateTime; // data e hora
```

Tipo	Declaração	Exemplo de valor
Literal (texto)	String	'Olá Mundo'
Inteiro	Integer	42
Real	Float / Double	3.14
Objeto	OleVariant	Referência a planilha, query...
Data/Hora	TDateTime	EncodeDate(2024,01,15)

Operadores e Expressões

Operadores Aritméticos

- + Adição
- - Subtração
- * Multiplicação
- / Divisão
- mod Resto da divisão

Operadores Relacionais

- = Igual a
- <> Diferente de
- < Menor que
- > Maior que
- <= Menor ou igual
- >= Maior ou igual

Operadores Lógicos

- AND E lógico
- OR OU lógico
- NOT Negação

Exemplo completo

```
var total: double;  
total := (preco * quantidade) - desconto;  
if (total > 0) AND (NOT cancelado) then ...
```

Arquitetura do Projeto no TestComplete

Estrutura de Pastas Recomendada

Organize o projeto para facilitar manutenção e reuso de código:

```
SuiteLogistica/  
├── AutomacaoFolder/                                ← Um projeto por aplicação  
│   ├── Script/  
│   │   ├── Lib/  
│   │   │   ├── Database.sd      ← Funções de banco  
│   │   │   ├── Pages.sd        ← Funções de tela  
│   │   │   └── TestUtils.sd     ← Utilitários gerais  
│   │   └── Cenarios/  
│   │       ├── CT_01.sd  
│   │       └── CT_02.sd  
│   └── DDT/  
│       └── planilhaExemplo.xlsx ← Dados de teste
```

Regra de ouro: uma Suite por SQUAD → um Projeto por Rotina → uma aba por cenário de dados.

Mapeamento: Modo Flat vs Modo Tree

O TestComplete pode representar o mesmo componente de duas formas. Escolha UMA e mantenha para todo o projeto.

Modo Tree (hierárquico)

```
procedure comModoTree();  
begin  
    Sys.Process('Prox1122')  
        .VCLObject('frmProxa_1122')  
        .VCLObject('WallpaperLogin')  
        .VCLObject('Panel1')  
        .VCLObject('btn14')  
        .Click;  
end;
```

Modo Flat (recomendado)

```
procedure comModoFlat();  
begin  
    Sys.Process('Prox1122')  
        .VCLObject('frmPrincipal_1122')  
        .VCLObject('btn14')  
        .Click;  
end;
```

- Flat é mais simples, menos suscetível a quebras.
- Os dois modos NÃO são compatíveis — defina no início do projeto.
- Mudar depois exige reescrever TODO o mapeamento.

Introdução ao DelphiScript

Convenções de Nomenclatura

Elemento	Regra	Exemplo
Arquivo	PascalCase + extensão minúscula	TelaPrincipal.sd
Procedimento	Verbo imperativo + PascalCase	GravarPedido()
Função	Verbo + PascalCase	CalcularTotal()
Variável	camelCase	valorTotal, codProd
Constante	UPPER_SNAKE_CASE	FONTE_DADOS
Parâmetro	Prefixo A + PascalCase	ANumPedido, APreco

```
procedure GravarPedido(ANumPedido: string; ATotal: double);  
var vlrFinal: double;  
const LIMITE_CREDITO = 10000;  
begin  
    vlrFinal := ATotal * 1.1; // acrescimo 10%
```

Comentários e Indentação

Código bem comentado e indentado é código que se mantém.

```
procedure ValidarPrecoPedido(ANumPed: string);
var origemPed: string; // Origem do pedido (1=Balcão, 2=Televendas)
begin
    // 1. Conectar ao banco
    qry := DatabaseLib.ConexaoBanco(FONTE_DADOS);

    // 2. Montar SQL
    qry.SQL := 'SELECT ORIGEMPED FROM PRODUTO WHERE NUMPROD = '' + ANumPed + ''';
    qry.Open;

    if not qry.IsEmpty then
    begin
        origemPed := qry.FieldName('ORIGEMPED').AsString;
        // 3. Validar
        if origemPed <> '1' then
        end;
    end;
```

Estrutura de Controle e Repetições

IF / ELSE — Decisão Condicional

Use IF para executar código apenas quando uma condição for verdadeira.

```
// IF simples
if condicao then
    comando;

// IF com bloco
if condicao then
begin
    comando1;
    comando2;
end;

// IF-ELSE completo
if valorTotal > LIMITE_CREDITO then
begin
    Log.Warning('Crédito excedido: ' + FloatToStr(valorTotal));
end
else
begin
    Log.Message('Crédito OK.');
```

FOR / WHILE — Repetição

```
// FOR – número fixo de repetições
for i := 1 to 10 do
begin
    Log.Message('Linha: ' + IntToStr(i));
end;

// WHILE – enquanto condição for verdadeira
while not planilha.EOF do
begin
    Log.Message(planilha.value('PRODUTO'));
    planilha.next; // IMPORTANTE: avança a linha!
end;

// Exemplo real: percorrer planilha e inserir no banco
planilha := DDT.ExcelDriver('DDTdados.xlsx', 'Pedidos');
while not planilha.EOF do
begin
    vSQL := 'INSERT INTO PCPEDC (NUMPED, CODCLI) VALUES (''
        + planilha.value('NUMPED') + '', '
        + planilha.value('CODCLI') + '');
```

⚠ *Sempre avance com .next dentro do WHILE para evitar loop infinito!*

Planilhas e Banco de Dados

AbrirPlanilha()

O que faz? Abre uma aba de planilha Excel usando a biblioteca DDT e retorna um objeto que representa os dados.

Parâmetros

caminho *string*
Caminho relativo da planilha Ex: 'DDT\planilha.xlsx'

worksheet *string*
Nome da aba Ex: 'Produtos'

Retorno: *OleVariant* (objeto DDT com os dados da planilha)

```
function AbrirPlanilha(caminho: string;  
                      worksheet: string): OleVariant;  
begin  
    result := DDT.ExcelDriver(caminho, worksheet);  
end;  
  
// Como usar:  
var pl: OleVariant;  
pl := AbrirPlanilha('DDT\dados.xlsx', 'Produtos');  
while not pl.EOF do
```

FecharPlanilha()

O que faz? Encerra a conexão DDT com a planilha e libera o recurso. Deve ser sempre chamada ao final do uso.

Parâmetros

oleDados *OleVariant*
O objeto retornado por AbrirPlanilha

⚠ Sempre feche a planilha após o uso — arquivo aberto impede que outros processos o leiam e gera erro na próxima execução.

```
procedure FecharPlanilha(oleDados: OleVariant);  
var Name: string;  
begin  
    DDT.CloseDriver(oleDados.Name);  
end;  
  
// Como usar:  
var pl: OleVariant;  
pl := AbrirPlanilha('DDT\dados.xlsx', 'Aba1');  
// ... usa os dados ...  
FecharPlanilha(pl); // SEMPRE ao final!
```

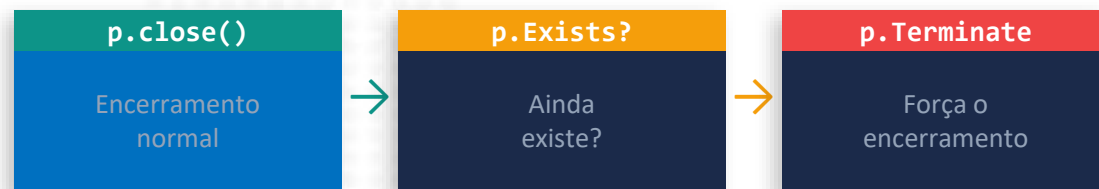
FecharProcesso()

O que faz? Encerra um processo pelo nome. Primeiro tenta fechar normalmente; se ainda existir, força o encerramento com Terminate.

Parâmetro

nome **string**
Nome do executável Ex: 'PCISIS1122'

Fluxo de encerramento



```

procedure FecharProcesso(nome: string);
var p;
begin
  Log.Message('Fechando: ' + nome);
  p := Sys.Process(nome);
  p.close;
  if (p.Exists) then
    p.Terminate; // força encerramento
end;
  
```

ConexaoBanco()

O que faz? Lê as credenciais do arquivo BD.ini e cria uma conexão ADO com o banco Oracle via ODBC. Suporta Query, Connection ou StoredProc.

Parâmetros

DBAtual **string**
Seção no BD.ini Ex: 'TESTELOCAL'

TipoConexao **string**
'Query' (padrão) | 'Connection' | 'Procedure'

Tipo de conexão criada (CASE)

'Query' → ADO.CreateADOQuery()

'Connection' → ADO.CreateConnection()

'Procedure' → ADO.CreateADOStoredProc()

```

qry := DatabaseLib.ConexaoBanco('TESTELOCAL');
qry.SQL := 'SELECT POSICAO FROM PRODT '
          + 'WHERE COD= ' + CODPRODUTO + ' ';
qry.Open;
if not qry.IsEmpty then
  
```

AceitaVendaFracionada()

O que faz? Aplica um UPDATE no banco preparando o produto para aceitar (ou não) venda em fração. Lê os dados direto da planilha.

SQL gerado

```
UPDATE PRODUTO
  SET ACEITAVENDA = <planilha.Value['ACEITAVENDAFRACAO']>
WHERE CODPROD = '<planilha.Value['CODPROD']>'
```

Código completo

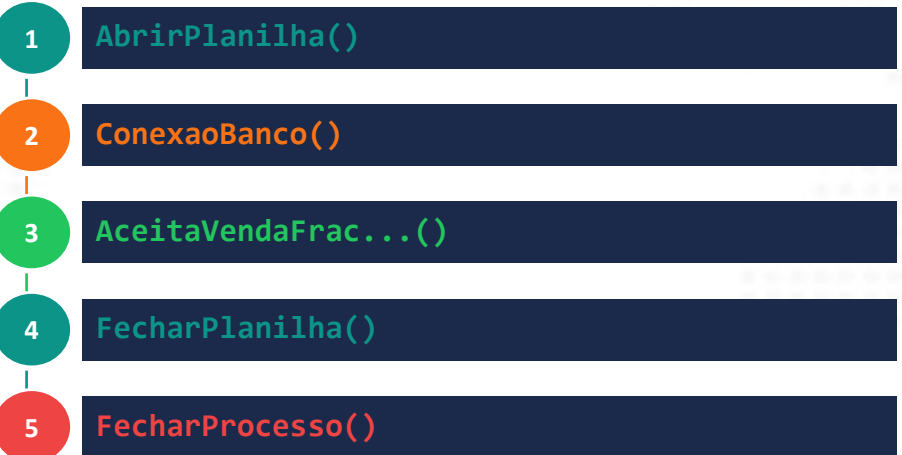
```
const DS_ALIAS = 'BDLOCAL';

procedure AceitaVenda(planilhaProduto: OleVariant);
var vsSQL: string;
    qry: OleVariant;
begin
  Log.Message('Alterando Aceita Venda ');
  planilhaProduto.first;           // volta ao 1º registro
  qry := DatabaseLib.ConexaoBanco(DS_ALIAS);
  vsSQL := 'UPDATE PCPRODUTO SET ACEITAVENDA = '
    + planilhaProduto.Value['ACEITAVENDA'];
  vsSQL := vsSQL + ' WHERE CODPRODUTO = ''
    + planilhaProduto.Value['CODPROD'] + ''';
  qry.SQL := vsSQL;
  qry.ExecSQL; // executa o UPDATE
```

Arquitetura & Fluxo

```
Suite_AUTOMACAO/
├── Script/
│   ├── Lib/
│   │   ├── DatabaseLib.sd ← ConexaoBanco
│   │   └── SistemaLib.sd  ← AbrirPlanilha
│   └── Cenarios/
│       └── CT_01.sd
└── DDT/
    └── Produtos.xlsx      ← planilhaProduto
```

Fluxo de execução do cenário



Trabalhando com Planilhas — Biblioteca DDT

A biblioteca DDT permite ler planilhas Excel (.xlsx) como fonte de dados nos seus cenários.

Método	Ação
planilha.close	Fecha a planilha e libera o recurso
planilha.next	Avança para a próxima linha
planilha.value('COL')	Retorna o valor da coluna indicada
planilha.EOF	TRUE quando não há mais linhas
planilha.first	Retorna ao início (primeira linha de dados)
planilha.name	Retorna o nome da aba ativa
DDT.CloseDriver(nome)	Fecha a conexão DDT pelo nome

```
caminho := 'DDTplanilhaExemplo.xlsx';  
planilha := DDT.ExcelDriver(caminho, 'AbaTeste');
```

Armadilhas Comuns com Planilhas

⚠ Células em branco

Uma célula aparentemente vazia pode conter espaço ou formatação invisível — o TestComplete lança erro. Apague a linha/coluna afetada com clic-direito → Excluir.

⚠ Float com vírgula

Oracle exige ponto como separador decimal (3.96, não 3,96). Sempre formate valores float na planilha como 3.96 ou trate no código.

⚠ String vs Número

Adicione apóstrofo (') antes do valor na célula para forçar texto. Ex: '001 → lido como string '001' e não como número 1.

⚠ Planilha aberta

Feche a planilha antes de executar o teste (ou abra em modo compartilhamento). O TestComplete não consegue ler um arquivo bloqueado por outro processo.

Conexão com Banco de Dados — ODBC Oracle

Pré-requisitos: instale ODAC 64 bits e configure a fonte ODBC antes de qualquer script.

- 1 Painel de Controle → pesquise ODBC → Configurar fontes de dados (64 bits)
- 2 Aba 'DSN de Sistema' → clicar em Adicionar
- 3 Selecionar driver 'Oracle em OraClient12Home1' → Concluir
- 4 Preencher: Data Source Name, TNS Service Name, UserID
- 5 DESMARCAR 'Enable Query Timeout' (aba Oracle) → Test Connection → OK

⚠ Desativar 'Enable Query Timeout' evita o erro ORA-01013 em pacotes de testes longos.

Código: Conexão e Consulta ao Banco

```
// — CONEXÃO —  
function CriarConexao(ACaminho: string): OleVariant;  
var cfg: OleVariant;  
    sid, uid, pwd, conn: string;  
    qry: OleVariant;  
begin  
    cfg := Storages.INI(ACaminho);  
    sid := cfg.GetSubSection('LOCAL').GetOption('Dsn', 'LOCAL');  
    uid := cfg.GetSubSection('LOCAL').GetOption('uid', 'LOCAL');  
    pwd := cfg.GetSubSection('LOCAL').GetOption('pwdDB', 'LOCAL');  
    conn := 'Dsn=' + sid + ';uid=' + uid + ';Password=' + pwd;  
    qry := ADO.CreateADOQuery();  
    qry.ConnectionString := conn;  
    result := qry;  
end;  
  
// — CONSULTA —  
procedure ValidarPedido(ANumPed: string);  
var qry: OleVariant;  
    vSQL, posicao: string;  
begin  
    vSQL := 'SELECT POSICAO FROM PRODUTO WHERE NUMPRO = '' + ANumPed + ''';  
    qry := CriarConexao('configBD.ini');  
    qry.SQL := vSQL;  
    qry.Open;
```


Referência: Métodos da Query e da Planilha

Método / Prop.	Tipo	Descrição
qry.Open	Ação	Executa SELECT e abre o cursor
qry.Close	Ação	Fecha o cursor e libera recursos
qry.ExecSQL	Ação	Executa INSERT / UPDATE / DELETE
qry.SQL	Prop.	String SQL a ser executada
qry.IsEmpty	Prop.	TRUE se nenhuma linha retornou
qry.EOF	Prop.	TRUE ao chegar na última linha
qry.first	Ação	Volta para o primeiro registro
qry.Next	Ação	Avança para o próximo registro
qry.FieldName('COL').AsString	Prop.	Retorna valor do campo como texto
qry.FieldName('COL').Value	Prop.	Retorna valor bruto do campo

Métodos & Propriedades do TestComplete

SetText vs Keys — Inserindo Valores

Ambas inserem texto em um campo, mas funcionam de forma diferente:

SetText

```
campo.SetText('12345');  
// "empurra" o valor inteiro de uma vez
```

- Rápido — ideal para campos simples.
- Pode não disparar eventos de digitação.
- Recomendado para IDs, códigos, senhas.

Keys

```
campo.Keys('12345');  
// simula cada tecla pressionada
```

- Dispara eventos de teclado (onChange, onKeyPress).
- Use quando o sistema valida a digitação.
- Necessário para campos com máscara.

Combinações especiais com Keys

```
campo.Keys('[Enter]'); // pressiona Enter  
campo.Keys('[Tab]');   // pressiona Tab  
campo.Keys('^a');       // Ctrl+A (selecionar tudo)  
campo.Keys('^[Del]');   // Ctrl+Delete  
campo.Keys('[F4]');     // tecla F4  
campo.Keys('[Esc]');    // tecla Escape  
campo.Keys('[P500]');   // pausa 500 ms
```

Referência: Teclas Especiais com Keys()

Código	Tecla	Código	Tecla
^	Ctrl	[Enter]	Enter
!	Shift	[Esc]	Escape
~	Alt	[Tab]	Tab
[BS]	Backspace	[Del]	Delete
[F1]...[F12]	F1 até F12	[Home]	Home
[End]	End	[Up][Down]	Setas
[Left][Right]	Setas	[PgUp][PgDn]	Page Up/Down
[Pnnn]	Pausa nnn ms	[Win]	Tecla Windows

```
// Exemplos práticos
Sys.Process('APP').VCLObject('Edit1').Keys('^a[Del]'); // seleciona tudo e apaga
Sys.Process('APP').VCLObject('BtnSalvar').Keys('[Enter]');
Sys.Process('APP').VCLObject('Edit1').Keys('[P300]12345'); // pausa 300ms antes
```

Propriedades de Estado dos Objetos

Propriedade	Retorno	Quando usar
Enabled	Boolean	Verificar se o campo está habilitado para interação
Visible	Boolean	Verificar se o controle está visível na tela (pode estar fora da área visível)
VisibleOnScreen	Boolean	Verificar se qualquer parte do objeto está visível e clicável na tela
Exists	Boolean	Verificar se o objeto foi encontrado (evita erros em objetos opcionais)
wText	String	Ler o texto contido em um campo, label ou botão
wItemCount	Integer	Contar itens de um combo/listbox
Checked	Boolean	Verificar se checkbox / radio está marcado
Focused	Boolean	Verificar se o objeto está com foco do teclado

```
// Uso prático
var btn: OleVariant;
btn := Sys.Process('APP').VCLObject('btnSalvar');
if btn.Enabled then btn.Click
else Log.Warning('Botão Salvar desabilitado!');
```

WaitProperty — Aguardar Estado de um Objeto

Use WaitProperty para sincronizar seu script com o tempo de resposta da aplicação.

```
// Sintaxe
objeto.WaitProperty(NomePropriedade, ValorEsperado, TempoLimiteMs)
// Retorna TRUE se atingido, FALSE se timeout

// — Exemplos práticos —————

var btn: OleVariant;
btn := Sys.Process('APP').VCLObject('btnSalvar');

// 1. Aguarda o botão ficar habilitado por até 5 segundos
if btn.WaitProperty('Enabled', true, 5000) then
    btn.Click
else
    Log.Error('Botão Salvar não habilitou no tempo esperado');

// 2. Aguarda campo ficar visível
if campo.WaitProperty('VisibleOnScreen', true, 3000) then
    campo.SetText('12345');

// 3. Aguarda mensagem de sucesso aparecer
if msgLabel.WaitProperty('wText', 'Gravado com sucesso!', 8000) then
    Log.Checkpoint('Gravação confirmada na tela');
```

Dica: Prefira WaitProperty a Delay() — é mais inteligente e não desperdiça tempo.

Localizar Objetos: Find, FindChild e Exists

```
// FindChild – busca um filho com propriedades específicas
var grid: OleVariant;
grid := Sys.Process('APP').VCLObject('frmPrincipal')
        .FindChild('WndClass', 'TDBGrid', 5);

// FindChild com múltiplas propriedades
var btn: OleVariant;
btn := Sys.Process('APP').VCLObject('frmPrincipal')
        .FindChild(['WndClass', 'wText'], ['TButton', 'Salvar'], 5);

// Verificar se o objeto existe antes de usar
if grid.Exists then
    Log.Message('Grid encontrado: ' + grid.FullName)
else
    Log.Warning('Grid não encontrado na tela');

// WaitVCLObject – como VCLObject mas aguarda o objeto surgir
var frm: OleVariant;
frm := Sys.Process('APP').WaitVCLObject('frmCadastro', 5000);
```

- Existe retorna FALSE (não gera erro) — sempre mais seguro que VCLObject direto.
- O terceiro parâmetro de FindChild é a profundidade de busca (recomendado: 5).
- Use WaitVCLObject quando uma janela abre de forma assíncrona.

Melhorando o Log do Teste

Função	Ícone	Quando usar
Log.Message('texto')		Mensagens informativas gerais
Log.Error('texto')		Falha crítica — marca o teste como FAILED
Log.Warning('texto')		Alerta — não falha o teste
Log.Checkpoint('texto')		Validação bem-sucedida
Log.Event('texto')		Evento relevante do fluxo
Log.Link('url','rótulo')		Adiciona link (ex: caso no Jira)
Log.Picture(objeto,'nome')		Captura screenshot do objeto
Log.LockEvents()		Suprime eventos de clique/teclado do log

```
Log.LockEvents(); // No início do script – elimina ruído de eventos no log
Log.Checkpoint('Pedido ' + numPed + ' gravado com sucesso – posição: L');
Log.Error('CONDVENDA divergente – esperado: 1, recebido: ' + condvenda);
Log.Link('https://jira.empresa.com/CT-99', 'CT-99 – Caso de Teste');
```

Debug — Depurando seu Script

F11

Executa linha por linha — entra dentro das funções chamadas.

F10

Executa linha por linha — NÃO entra dentro das funções.

F5

Continua a execução até a próxima marcação de debug ou fim.

Ctrl+F10

Mostra o valor da variável selecionada no momento.

Ctrl+F12

Abre a janela Evaluate — avalia qualquer expressão.

Como criar uma marcação de debug:

Boas Práticas & Encerramento

Configurações Essenciais do TestComplete

Desativar Stop on Error

Tools → Default Project Properties → Playback → desmarcar 'Stop on error'. Erros menores não interrompem o fluxo.

Modo Flat obrigatório

Para projetos novos, configure o mapeamento em Modo Flat desde o início. Não mescle modos.

Limitar Logs

Tools → Options → Log → desmarcar 'Store all logs' → definir 5 logs recentes. Evita projeto pesado.

Remover variáveis aux.

Tools → Default Project Properties → Recording → desabilitar criação automática de variáveis auxiliares.

Remover fotos no Record

Tools → Default Project Properties → Visualizer → 'Collect Test Visualizer data during recording' → Off.

Remover Timeout ODBC

Administrador ODBC 64 bits → DSN de Sistema → Configurar → aba Oracle → desmarcar Enable Query Timeout.

Problemas Comuns e Soluções

Problema: Aplicativos Windows abrem ao fechar o TC

Solução: 1. Desativar serviço Superfetch 2. Iniciar → Configurações → Privacidade → Aplicativos em segundo plano → Desligar 3. Tools → Options → Engines → desmarcar suporte Windows Store

Problema: ORA-01013: timeout na consulta

Solução: Desmarque 'Enable Query Timeout' no driver ODBC 64 bits (aba Oracle). Reinicie o serviço ODBC após a alteração.

Problema: Erro de arquitetura 32/64 bits

Solução: ODBC, Excel e TestComplete devem ter a MESMA arquitetura (todos 32 ou todos 64 bits). Misturá-las gera incompatibilidade de driver.

Problema: Planilha travada / não abre

Solução: Certifique-se que o arquivo .xlsx está fechado antes de executar o script, ou abra-o em modo de compartilhamento (Pasta de Trabalho Compartilhada).

Checklist de Qualidade — Antes de Commitar

- ✓ Script executa do início ao fim sem intervenção manual
- ✓ Planilha DDT está fechada (ou em modo compartilhamento)
- ✓ Log usa Checkpoint para validações e Error para falhas
- ✓ Log.LockEvents() está no início do script
- ✓ Não há variáveis não-utilizadas ou código morto
- ✓ Nomes de procedimentos, arquivos e variáveis seguem as convenções
- ✓ Valores Float usam ponto decimal (3.14 e não 3,14)
- ✓ Strings numéricas (ex: código de filial '001') têm apóstrofo na planilha
- ✓ Células em branco foram verificadas na planilha
- ✓ Commit realizado com mensagem descritiva no padrão Conventional Commits

Parabéns!

Você chegou ao fim do curso.

- ✓ Git: histórico, branches, pull requests
- ✓ Lógica: variáveis, operadores, estruturas de controle
- ✓ Arquitetura: suites, projetos, bibliotecas, DDT
- ✓ DelphiScript: convenções, comentários, indentação
- ✓ Planilhas & Banco: DDT, ODBC, queries Oracle
- ✓ TC: Keys, SetText, propriedades, WaitProperty, Log

Agora é hora de praticar. Bons testes! 🚀

NobleProg

Obrigado por participar!

Esperamos que este curso tenha sido de grande valor para você e sua organização.

Você tem alguma dúvida ou gostaria de mais informações sobre nossos cursos?

SEU FEEDBACK

FAZ A DIFERENÇA PARA NÓS!



OBRIGADO!!
NobleProg

www.nobleprog.com